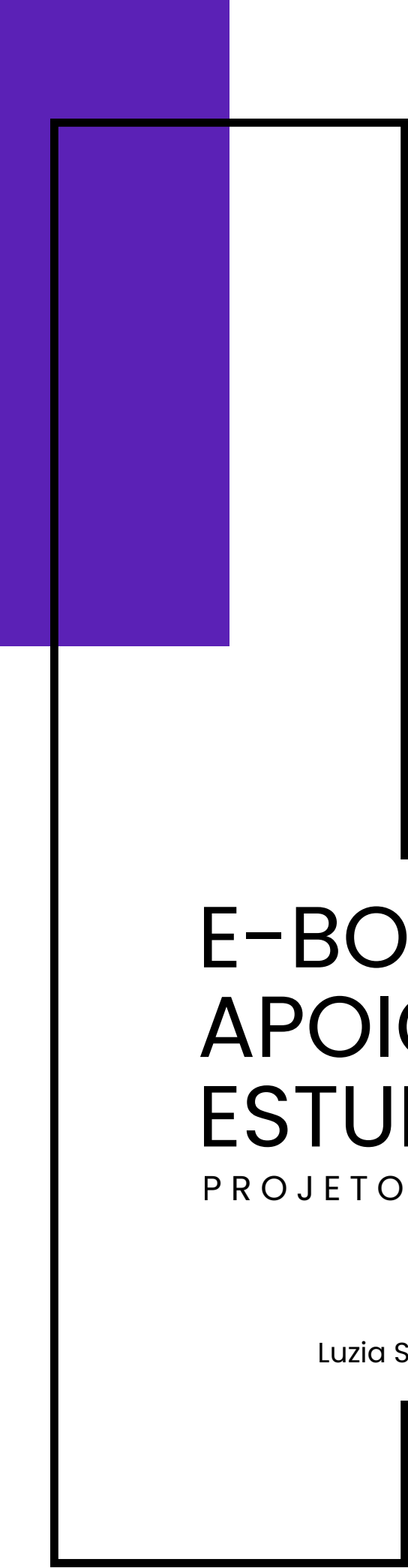




E-BOOK DE APOIO AOS ESTUDOS

PROJETO GIRLS TECH

Luzia Santos

Sumário

Parte 1 — Semana 1: Fundamentos, lógica e estruturas

1. Introdução ao Python
2. Variáveis, tipos e operadores
3. Condicionais
4. Loops
5. Listas, dicionários e tuplas
6. Funções

Parte 2 — Semana 2: Arquivos, POO, bibliotecas e projeto

7. Manipulação de arquivos
8. Tratamento de exceções
9. Módulos e pacotes
10. Programação Orientada a Objetos
11. Bibliotecas do ecossistema Python
12. Automação e manipulação de dados
13. Projeto prático final

Parte 3 — IA como apoio à programação

14. Como usar IA para aprender a programar

Parte 4 — Lista de Exercícios

15. Semana 1 — Fundamentos, lógica e estruturas
16. Semana 2 — Arquivos, POO, bibliotecas e IA

Parte 1 — Semana 1: Fundamentos, lógica e estruturas

1. Introdução ao Python

Python é uma linguagem de programação de alto nível, criada por Guido van Rossum em 1991, conhecida pela sintaxe simples e legível. É usada em automação, desenvolvimento web, ciência de dados, inteligência artificial e muito mais — o que a torna uma ótima porta de entrada para quem está começando.

Por que Python é uma boa primeira linguagem?

- Sintaxe próxima da linguagem natural (inglês)
- Comunidade enorme — quase todo erro já foi resolvido em algum fórum
- Usada em áreas muito diferentes, então o conhecimento se aplica a vários caminhos de carreira

Instalando e rodando Python

Para praticar fora da aula, duas opções simples:

- **Replit** ou **Google Colab** — rodam no navegador, sem instalar nada
- **Instalação local** — baixar em python.org e usar um editor como VS Code

Para testar rapidamente um trecho de código, o **modo interativo** (REPL) é útil: basta digitar `python` no terminal e testar linha a linha.

Comentários

```
# Isso é um comentário de uma linha
"""
Isso é um comentário
de várias linhas
"""
```

Comentários não são executados — servem para explicar o código para você mesma no futuro, ou para outras pessoas lendo.

2. Variáveis, tipos e operadores

Variáveis

Uma variável é um espaço na memória que guarda um valor, identificado por um nome.

```
nome = "Ana"
idade = 25
altura = 1.68
estudante = True
```

Em Python, não é preciso declarar o tipo — ele é inferido automaticamente a partir do valor. Isso é chamado de **tipagem dinâmica**.

Regras para nomes de variáveis:

- Podem conter letras, números e `_`, mas não podem começar com número
- São case-sensitive (`idade` e `Idade` são variáveis diferentes)
- Por convenção, usam-se nomes em `snake_case` (minúsculas com underscore):
`nome_completo`, `data_nascimento`

Tipos primitivos

Tipo	Exemplo	Descrição
<code>int</code>	<code>10</code> , <code>-3</code>	números inteiros
<code>float</code>	<code>3.14</code> , <code>-0.5</code>	números decimais
<code>str</code>	<code>"olá"</code>	texto (cadeia de caracteres)
<code>bool</code>	<code>True</code> , <code>False</code>	valores lógicos

Para verificar o tipo de uma variável:

```
print(type(idade))    #<class 'int'>
```

Conversão de tipos

```
idade_texto = "25"
idade_numero = int(idade_texto)    #str -> int
preco = float("19.90")             #str -> float
texto = str(42)                    #int -> str
```

Isso é muito comum quando se recebe dados do usuário com `input()`, que sempre retorna texto (`str`):

```
idade = int(input("Digite sua idade: "))
```

Operadores aritméticos

Operador	Significado	Exemplo
<code>+</code>	soma	<code>5 + 2</code> → 7
<code>-</code>	subtração	<code>5 - 2</code> → 3
<code>*</code>	multiplicação	<code>5 * 2</code> → 10
<code>/</code>	divisão (sempre retorna float)	<code>5 / 2</code> → 2.5
<code>//</code>	divisão inteira	<code>5 // 2</code> → 2
<code>%</code>	resto da divisão (módulo)	<code>5 % 2</code> → 1
<code>**</code>	potência	<code>5 ** 2</code> → 25

O operador `%` é muito usado para verificar se um número é par ou ímpar: `numero % 2 == 0` é `True` quando o número é par.

Operadores de comparação

`==` (igual), `!=` (diferente), `>`, `<`, `>=`, `<=` — todos retornam `True` ou `False`.

Atenção: `=` atribui um valor, `==` compara. É um dos erros mais comuns de quem está começando.

Operadores lógicos

- `and` — verdadeiro se ambos os lados forem verdadeiros
- `or` — verdadeiro se pelo menos um lado for verdadeiro
- `not` — inverte o valor lógico

```
idade = 20
tem_documento = True
pode_votar = idade >= 16 and tem_documento
```

f-strings (formatação de texto)

A forma mais moderna e legível de inserir variáveis dentro de um texto:

```
nome = "Ana"
idade = 25
print(f"{nome} tem {idade} anos")
```

Exercício de fixação: peça ao usuário nome e ano de nascimento, calcule a idade aproximada e exiba uma frase usando f-string.

3. Condicionais

Condicionais permitem que o programa tome decisões diferentes dependendo de uma condição.

```
idade = 20

if idade >= 18:
    print("Maior de idade")
elif idade >= 12:
    print("Adolescente")
else:
    print("Criança")
```

Como o Python decide os blocos: ao contrário de outras linguagens que usam `{ }`, Python usa **indentação** (geralmente 4 espaços) para definir o que pertence a cada bloco. Isso obriga o código a ser organizado, mas exige atenção — um erro de indentação quebra o programa.

Condicionais aninhadas

```
nota = 7.5
frequencia = 85

if frequencia >= 75:
    if nota >= 6:
        print("Aprovado")
    else:
        print("Reprovado por nota")
else:
    print("Reprovado por falta")
```

Operador ternário

Uma forma compacta de escrever um `if/else` simples em uma linha:

```
status = "Aprovado" if nota >= 6 else "Reprovado"
```

`match` (Python 3.10+)

Uma alternativa ao clássico `switch/case` de outras linguagens, útil quando há muitas comparações do mesmo valor:

```
dia = 3
match dia:
    case 1:
        print("Segunda")
    case 2:
        print("Terça")
    case 3:
        print("Quarta")
    case _:
        print("Outro dia")
```

`case _` funciona como o “caso padrão” (default).

Exercício de fixação: escreva um programa que classifica um número digitado em positivo, negativo ou zero, e também informa se é par ou ímpar.

4. Loops

Loops repetem um bloco de código várias vezes, evitando repetir código manualmente.

for

Usado quando se sabe (ou se pode calcular) quantas vezes o loop deve rodar, ou quando se está percorrendo uma coleção de itens.

```
for i in range(5):           #0, 1, 2, 3, 4
    print(i)

frutas = ["maçã", "banana", "uva"]
for fruta in frutas:
    print(fruta)
```

`range()` tem três formas de uso:

```
range(5) # 0 a 4
range(2, 5) # 2 a 4
range(0, 10, 2) # 0, 2, 4, 6, 8 (pula de 2 em 2)
```

while

Usado quando não se sabe de antemão quantas repetições serão necessárias — o loop roda enquanto uma condição for verdadeira.

```
contador = 0
while contador < 3:
    print(contador)
    contador += 1
```

Cuidado com loops infinitos: se a condição do `while` nunca se tornar falsa, o programa nunca para. Sempre garanta que algo dentro do loop eventualmente muda a condição.

`break` e `continue`

- `break` — interrompe o loop imediatamente
- `continue` — pula para a próxima repetição, sem executar o resto do bloco

```
for numero in range(10):
    if numero == 5:
        break          # para o loop ao chegar em 5
    if numero % 2 == 0:
        continue       # pula os pares, só imprime ímpares
    print(numero)
```

Loops aninhados

Um loop dentro do outro — útil para tabelas, matrizes e combinações:

```
for i in range(3):
    for j in range(3):
        print(f"({i}, {j})")
```

Exercício de fixação: escreva um programa que imprime a tabuada de um número digitado pelo usuário (de 1 a 10), usando `for`.

5. Listas, dicionários e tuplas

Listas

Coleções ordenadas e **mutáveis** (podem ser alteradas depois de criadas).

```
frutas = ["maçã", "banana", "uva"]

frutas.append("pera")          #adiciona no final
frutas.remove("banana")       #remove um item específico
frutas.insert(0, "morango")    #insere em uma posição
frutas.pop()                   #remove e retorna o último item

print(frutas[0])               #primeiro item
print(frutas[-1])              #último item
print(frutas[1:3])             #slicing: itens do índice 1 até o 2
print(len(frutas))             #tamanho da lista
```

List comprehension — forma compacta de criar listas a partir de outra:

```
numeros = [1, 2, 3, 4, 5]
pares = [n for n in numeros if n%2==0]           #[2, 4]
dobrados = [n * 2 for n in numeros]             #[2, 4, 6, 8, 10]
```

Dicionários

Coleções de pares **chave: valor**, muito usadas para representar objetos do mundo real.

```
peessoa = {"nome": "Ana", "idade": 25, "cidade": "Osasco"}

print(peessoa["nome"])
print(peessoa.get("profissao", "não informado"))      #evita erro se a chave não exist

peessoa["profissao"] = "Analista"                     #adiciona ou atualiza uma chave

for chave, valor in peessoa.items():
    print(f'{chave}: {valor}')
```

`.get()` é mais seguro que `[]` porque não gera erro quando a chave não existe — em vez disso, retorna um valor padrão.

Tuplas

Coleções ordenadas e **imutáveis** (não podem ser alteradas após a criação).

```
coordenadas = (10, 20)
x, y = coordenadas # desempacotamento
```

Quando usar cada uma:

- Lista → coleção que vai crescer, encolher ou mudar (carrinho de compras, lista de tarefas)
- Tupla → dados fixos que não devem mudar (coordenadas, configurações)
- Dicionário → quando cada valor tem um “nome” (dados de uma pessoa, configurações nomeadas)

Exercício de fixação: crie um dicionário representando um produto (nome, preço, quantidade em estoque) e escreva um programa que calcula o valor total em estoque.

6. Funções

Funções organizam o código em blocos reutilizáveis, evitando repetição.

```
def calcular_media(notas):  
    return sum(notas) / len(notas)  
  
media = calcular_media([8, 7, 9])  
print(media)
```

Parâmetros com valor padrão

```
def saudacao(nome, cumprimento="Olá"):  
    return f'{cumprimento}, {nome}!'  
  
print(saudacao("Ana"))           #Olá, Ana!  
print(saudacao("Ana", "Bom dia")) #Bom dia, Ana!
```

args** e *kwargs**

Para funções que recebem um número variável de argumentos:

```
def somar_tudo(*numeros):  
    return sum(numeros)  
  
somar_tudo(1, 2, 3, 4)    #10
```

Escopo de variáveis

Uma variável criada dentro de uma função só existe dentro dela (escopo local), a não ser que seja explicitamente retornada:

```
def exemplo():  
    x = 10 # variável local, não existe fora da função  
    return x  
  
resultado = exemplo()
```

Funções como valores

Em Python, funções podem ser atribuídas a variáveis, passadas como argumento e retornadas por outras funções — isso é a base de conceitos mais avançados usados em bibliotecas de dados e automação.

Exercício de fixação (fecha a Semana 1): escreva uma função `analisar_notas(notas)` que recebe uma lista de números e retorna um dicionário com a lista de notas pares, ímpares e a média. Esse exercício usa `for`, `if`, lista, dicionário e função — tudo que foi visto até aqui.

Parte 2 — Semana 2: Arquivos, POO, bibliotecas e projeto

7. Manipulação de arquivos

Programas frequentemente precisam ler dados de um arquivo ou salvar resultados.

```
# Escrever em um arquivo
with open("dados.txt", "w") as arquivo:
    arquivo.write("Primeira linha\n")
    arquivo.write("Segunda linha\n")

# Ler um arquivo
with open("dados.txt", "r") as arquivo:
    conteudo = arquivo.read()
    print(conteudo)

# Ler linha por linha
with open("dados.txt", "r") as arquivo:
    for linha in arquivo:
        print(linha.strip()) # strip() remove quebras de linha extras
```

O `with` garante que o arquivo seja fechado automaticamente, mesmo se ocorrer um erro — é a forma recomendada de trabalhar com arquivos em Python.

Modos de abertura comuns: `"r"` (leitura), `"w"` (escrita, sobrescreve o arquivo), `"a"` (append, adiciona ao final).

Trabalhando com CSV

CSV é um formato de arquivo muito comum para dados tabulares (planilhas simples):

```
import csv

with open("alunos.csv", "r") as arquivo:
    leitor = csv.reader(arquivo)
    for linha in leitor:
        print(linha)
```

8. Tratamento de exceções

Erros acontecem — um arquivo pode não existir, o usuário pode digitar texto onde se esperava número. O tratamento de exceções evita que o programa quebre de forma abrupta.

```
try:
    numero = int(input("Digite um número: "))
    resultado = 10 / numero
    print(resultado)
except ValueError:
    print("Isso não é um número válido")
except ZeroDivisionError:
    print("Não é possível dividir por zero")
else:
    print("Nenhum erro ocorreu")
finally:
    print("Fim do programa")
```

- **try** — bloco onde o erro pode ocorrer
- **except** — o que fazer se o erro acontecer (pode ser específico por tipo de erro)
- **else** — executa apenas se não houve erro
- **finally** — sempre executa, com ou sem erro (útil para fechar conexões, arquivos, etc.)

Boas práticas: capturar o tipo específico de exceção (`ValueError`, `ZeroDivisionError`) é melhor do que capturar tudo genericamente com `except:`, porque evita esconder erros inesperados que deveriam ser corrigidos.

9. Módulos e pacotes

Um **módulo** é simplesmente um arquivo Python (`.py`) que pode ser importado em outro. Isso permite organizar código em partes reutilizáveis.

```
# arquivo utils.py
def somar(a, b):
    return a + b
```

```
# outro arquivo
import utils
print(utils.somar(2, 3))

# ou importar só o necessário
from utils import somar
print(somar(2, 3))
```

Módulos da biblioteca padrão

Python já vem com muitos módulos prontos:

```
import math
print(math.sqrt(16))           #4.0

import random
print(random.randint(1, 10))   #número aleatório entre 1 e 10

import datetime
print(datetime.date.today())
```

Um **pacote** é uma pasta com vários módulos organizados juntos (é assim que bibliotecas maiores, como **pandas**, são estruturadas internamente).

10. Programação Orientada a Objetos

POO organiza o código em torno de **objetos** que combinam dados (atributos) e comportamentos (métodos) — útil para representar entidades do mundo real de forma organizada.

Classes e objetos

```
class Pessoa:
    def __init__(self, nome, idade):
        self.nome = nome
        self.idade = idade

    def apresentar(self):
        return f"Olá, meu nome é {self.nome} e tenho {self.idade} anos"

ana = Pessoa("Ana", 25)
print(ana.apresentar())
```

- `class` define o “molde” do objeto
- `__init__` é o **construtor** — roda automaticamente ao criar o objeto, e define seus atributos iniciais
- `self` representa o próprio objeto sendo criado ou usado
- `ana` é uma **instância** da classe `Pessoa`

Atributos e métodos

```
class ContaBancaria:
    def __init__(self, titular, saldo=0):
        self.titular = titular
        self.saldo = saldo

    def depositar(self, valor):
        self.saldo += valor

    def sacar(self, valor):
        if valor > self.saldo:
            print("Saldo insuficiente")
        else:
            self.saldo -= valor

conta = ContaBancaria("Ana")
conta.depositar(100)
conta.sacar(30)
print(conta.saldo) # 70
```

Herança (visão geral)

Herança permite que uma classe reaproveite atributos e métodos de outra:

```
class ContaPoupanca(ContaBancaria):
    def render_juros(self, taxa):
        self.saldo += self.saldo * taxa
```

`ContaPoupanca` herda tudo de `ContaBancaria` e adiciona um método novo. Esse é um tópico mais avançado — vale entender a ideia geral, sem precisar dominar todos os detalhes na primeira passada.

11. Bibliotecas do ecossistema Python

Bibliotecas são conjuntos de código pronto que resolvem problemas comuns, evitando reinventar a roda.

Instalação:

```
pip install nome-da-biblioteca
```

`requests` — requisições web e automação

```
import requests

resposta = requests.get("https://api.exemplo.com/dados")
print(resposta.status_code)
print(resposta.json())
```

Muito usada para consumir APIs e automatizar tarefas que envolvem a internet.

`pandas` — manipulação de dados

```
import pandas as pd

df = pd.read_csv("dados.csv")
print(df.head())           # primeiras linhas
print(df["coluna"].mean())  # média de uma coluna
df_filtrado = df[df["idade"] > 18]
```

`pandas` é a biblioteca mais usada para análise de dados em Python — organiza dados em tabelas (DataFrames) fáceis de filtrar, agrupar e calcular.

Outras bibliotecas úteis de citar

- `datetime` — datas e horários
 - `os` — interação com o sistema operacional (arquivos, pastas)
 - `matplotlib` / `seaborn` — gráficos e visualização de dados
-

12. Automação e manipulação de dados

Automação é aplicar tudo que foi aprendido para eliminar tarefas repetitivas. Exemplos comuns para praticar:

- Renomear vários arquivos de uma pasta automaticamente
- Ler um arquivo CSV, filtrar e calcular estatísticas
- Buscar informações em uma API e salvar em um arquivo
- Enviar um relatório automático por e-mail em horários definidos

Exemplo simples — organizar arquivos por extensão:

```
import os
import shutil

pasta = "downloads"
for arquivo in os.listdir(pasta):
    extensao = arquivo.split(".")[-1]
    pasta_destino = os.path.join(pasta, extensao)
    os.makedirs(pasta_destino, exist_ok=True)
    shutil.move(os.path.join(pasta, arquivo), pasta_destino)
```

Esse tipo de script junta módulos (`os`, `shutil`), loops, condicionais e manipulação de arquivos — exatamente o conteúdo das duas semanas trabalhando junto.

13. Projeto prático final

Sugestão de projeto integrador, combinando tudo o que foi estudado:

Construa um programa de **gerenciamento de gastos pessoais** que:

1. Lê os gastos de um arquivo CSV (data, categoria, valor)
2. Usa uma classe `Gasto` para representar cada registro
3. Calcula o total gasto por categoria (usando `pandas` ou dicionários)
4. Trata erros caso o arquivo não exista ou tenha dados inválidos
5. Salva um resumo em um novo arquivo de saída

Esse projeto usa arquivos, POO, tratamento de exceções, bibliotecas e lógica — reunindo o conteúdo das duas semanas em algo com aplicação real.

Parte 3 — IA como apoio à programação

14. Como usar IA para aprender a programar

Ferramentas de IA (como ChatGPT, Claude, GitHub Copilot) podem acelerar o aprendizado — mas usadas do jeito errado, atrapalham mais do que ajudam.

IA como par de programação, não como atalho

O maior risco de usar IA para aprender é copiar e colar código sem entender o que ele faz. Isso cria uma sensação falsa de progresso: o exercício “funciona”, mas o conhecimento não fica.

Regra de ouro: nunca use um código que você não consegue explicar linha por linha.

Como perguntar melhor

Em vez de pedir “escreva um programa que faça X”, tente:

- **Pedir explicação de erro:** “Estou recebendo este erro: [colar o erro]. O que ele significa e por que está acontecendo no meu código?”
- **Pedir revisão, não substituição:** “Escrevi este código para resolver [problema]. Ele está funcionando, mas tem algo que pode ser melhorado?”
- **Pedir analogias:** “Explique o que é um dicionário em Python como se eu nunca tivesse programado antes”
- **Pedir casos de teste:** “Quais casos eu deveria testar nesta função para ter certeza de que ela está correta?”
- **Pedir para não dar a resposta pronta:** “Não me dê o código todo — me dê uma dica do próximo passo”

Boas práticas

- Sempre rode o código que a IA sugeriu e observe o que ele faz antes de aceitar
- Se a IA gerar algo que você não entende, pergunte “por que você escreveu desta forma?” antes de usar
- Use a IA para revisar seu próprio código, não só para gerar do zero
- Desconfie de respostas muito confiantes — IA pode errar, especialmente em detalhes específicos

Exercícios de fixação

Assistir a aula e entender a explicação é só a primeira parte do aprendizado — é na hora de escrever o código sozinha, sem ninguém guiando, que você descobre o que realmente aprendeu e o que ainda precisa fixar. As dúvidas que aparecem tentando resolver por conta própria valem mais do que qualquer explicação, porque é ali que o conteúdo vira conhecimento de verdade.

Programar é uma habilidade prática, como tocar um instrumento: ninguém aprende só assistindo alguém tocar. A lógica só vira intuição depois de repetida várias vezes com as próprias mãos. Por isso, faça os exercícios mesmo que erre de primeira — errar, debugar e corrigir é exatamente o processo que fixa o conteúdo e qualquer dúvida estamos aqui para ajudar.

Parte 4 — Lista de Exercícios

15. Semana 1 — Fundamentos, lógica e estruturas

Exercício 1 — Variáveis e tipos

Peça ao usuário nome, ano de nascimento e cidade. Calcule a idade aproximada (ano atual - ano de nascimento) e exiba uma frase usando f-string, por exemplo: "Ana, 25 anos, mora em Osasco".

Exercício 2 — Condicionais

Escreva um programa que recebe um número e informa: se é positivo, negativo ou zero; e se é par ou ímpar. Use if/elif/else e o operador %.

Exercício 3 — Loops

Peça um número ao usuário e imprima a tabuada completa dele (de 1 a 10), usando for e range(). Depois, reescreva o mesmo programa usando while.

Exercício 4 — Listas e dicionários

Crie um dicionário representando um produto (nome, preço, quantidade em estoque). Escreva um programa que calcula o valor total em estoque (preço * quantidade) e exibe o resultado formatado.

Exercício 5 — Funções (integrador)

Escreva uma função analisar_notas(notas) que recebe uma lista de números e retorna um dicionário com as notas pares, as notas ímpares e a média. Teste a função com uma lista de pelo menos 5 notas.

16. Semana 2 — Arquivos, POO, bibliotecas e IA

Exercício 6 — Arquivos

Crie um programa que salva uma lista de tarefas em um arquivo `tarefas.txt` (uma por linha). Depois, escreva outro trecho de código que lê o arquivo e imprime cada tarefa numerada.

Exercício 7 — Tratamento de exceções

Peça dois números ao usuário e divida o primeiro pelo segundo. Trate os erros possíveis: usuário digitar texto em vez de número (`ValueError`) e tentativa de dividir por zero (`ZeroDivisionError`), exibindo uma mensagem clara para cada caso.

Exercício 8 — Módulos e POO

Crie uma classe `ContaBancaria` com atributos `titular` e `saldo`, e métodos `depositar(valor)` e `sacar(valor)` (o saque deve verificar se há saldo suficiente). Depois, mova essa classe para um arquivo separado (`conta.py`) e importe-a no seu programa principal.

Exercício 9 — Bibliotecas e automação

Usando a biblioteca `os`, escreva um script que lista todos os arquivos de uma pasta e organiza-os em subpastas por extensão (por exemplo, todos os `.txt` numa pasta `txt/`, todos os `.png` numa pasta `png/`).

Exercício 10 (alternativo) — Cadastro simples + IA

Crie um programa que mantém uma lista de contatos (nome e telefone) usando uma lista de dicionários. O programa deve permitir adicionar um contato e listar todos os contatos cadastrados, usando uma função para cada ação. Depois, peça a uma IA para revisar seu código e sugerir uma melhoria — antes de aplicar, explique com suas próprias palavras por que ela funciona.